CS F213 — Object-Oriented Programming

Project Report — Part B

Smart Wallet Transaction Logger

with Real-Time Anomaly Detection

Group 2

Antariksha Deb	2023A7PS0004H
Deep Mitra	2023B5AA0670H

BITS Pilani, Hyderabad Campus

June 2026

Technology Stack

Java 21 · JDBC (SQLite dev / PostgreSQL prod) · Java Swing + FlatLaf · Maven · JUnit 5 · GitHub

GitHub: https://github.com/Jupiter-2003/wallet-transaction-logger-and-anomaly-detector.git

Contents

1 Problem Definition

1.1 Context

The BITS Pilani Hyderabad Campus annual cultural festival, **Atmos**, deploys a digital wallet system used by 5,000–6,000 attendees across 90+ events, multiple food vendors, and ticket counters — processing over Rs. 1.5 crore in wallet transactions over the festival duration. Prior to this project, the festival's technical team had no structured, queryable audit trail of wallet transactions and no automated fraud or anomaly detection. Administrators who needed to investigate a suspicious transaction had to write raw SQL against the PostgreSQL production database directly, which was slow, error-prone, and a security risk.

1.2 Inputs

- Live wallet transaction stream: transaction ID, user ID, vendor ID, amount (Rs.), timestamp, transaction type (`CREDIT` / `DEBIT` / `REFUND`)
- Historical transaction records: loaded from a CSV file (or directly seeded into the database via the DAO layer)
- Admin filter queries: user ID, date range, amount bounds, transaction type, flagged-only toggle
- Anomaly detection configuration: tunable thresholds loaded from `anomaly_config.properties` on the classpath at startup

1.3 Outputs

- Persistent JDBC-backed transaction log stored in SQLite (development) or PostgreSQL (production)
- Admin query interface: a filterable, sortable `JTable` with red-highlighted flagged rows and CSV-export capability

- Real-time anomaly alerts with machine-readable flag codes (`AMOUNT_SPIKE`, `HIGH_VELOCITY`, `DORMANT_SPIKE`, `DUPLICATE_ATTEMPT`, `UNUSUAL_HOUR`) and human-readable descriptions, persisted to an `anomaly_log` table
- Serialised per-user statistical profiles (`UserProfile`) written to disk on exit so detection baselines survive application restarts

1.4 Constraints & Success Metrics

Constraint	Target	Design Decision	Driven By It
Detection latency	≤ 200 ms/txn	`ConcurrentHashMap`, lock-free profile reads, no I/O on detection path	Real-time alerting during peak load
Admin query response	< 1 s for $\leq$10,000 rows	`PreparedStatement`, connection pool, `SwingWorker` off EDT	Usable investigation UI
Concurrent safety	Zero data corruption under load	`BlockingQueue` producer-consumer, synchronized profile updates	Multi-vendor simultaneous writes
Crash resilience	Zero crashes on malformed input	Custom checked exceptions, try-with-resources, CSV row-skip on error	Untrusted CSV exports from wallet backend
Detection precision	$\geq$ 80% on synthetic test data	Welford's online algorithm for adaptive per-user baselines	Avoiding alert fatigue for admins
Baseline persistence	Survives app restarts	Java serialisation via `ProfileStore` (`ObjectOutputStream`)	Multi-day festival operation

Table 1: Design constraints and the decisions that satisfy them.

2 System Architecture & Design

2.1 Package Structure

Package	Classes	Responsibility
`model`	`Transaction` (abstract), `DebitTransaction`, `CreditTransaction`, `RefundTransaction`, `AnomalyAlert`, `UserProfile`, `TransactionType`	Domain objects and value types
`dao`	`GenericDAO<T,ID>`, `TransactionDAO`, `AnomalyDAO`, `DatabaseConnectionPool`, `SchemaInitialiser`	JDBC persistence layer
`anomaly`	`AnomalyRule`, `AnomalyConfig`, `AnomalyDetectionEngine`, `ProfileStore`, `AmountSpikeRule`, `HighVelocityRule`, `DormantSpikeRule`, `DuplicateAttemptRule`, `UnusualHourRule`	Detection engine and rule strategy implementations
`pipeline`	`TransactionProducer`	Producer side of the `BlockingQueue` pipeline
`gui`	`WalletLoggerApp`, `MainFrame`, `TransactionTableModel`, `AlertTableModel`, `CsvIngestWorker`, `SimulateTransactionDialog`, `Theme`, `TableRenderers`, `StatCard`, `PillLabel`, `RoundedPanel`	Swing desktop UI
`exceptions`	`TransactionPersistenceException`, `InvalidTransactionException`, `AnomalyConfigException`	Custom checked exception hierarchy
`util`	`CsvLoader`	CSV file ingestion and row validation

Table 2: Package-level organisation of the codebase.

2.2 Data Flow

The end-to-end pipeline flows as follows:

1. A transaction source (CSV replay via `CsvIngestWorker`, or manual entry via `SimulateTransactionDialog`) calls `queue.put(transaction)`, placing `Transaction` objects onto a `LinkedBlockingQueue<Transaction>`.
2. The `AnomalyDetectionEngine` runs on a dedicated background thread. Its `run()` loop calls `queue.take()`, which blocks without spinning until a transaction is available.
3. For each dequeued transaction, the engine: (a) retrieves or creates the user's `UserProfile` from a `ConcurrentHashMap`; (b) evaluates all five `AnomalyRule` implementations; (c) collects any fired `Verdict` objects into a `PriorityQueue<AnomalyAlert>` ordered by severity; (d) persists the transaction via `TransactionDAO.insert()`; (e) persists each `AnomalyAlert` via `AnomalyDAO.insert()`; (f) notifies registered `AlertListener` instances (the Swing GUI); (g) updates the `UserProfile` statistics using Welford's algorithm.
4. The GUI (`MainFrameimplementsAlertListener`) receives callbacks on the engine thread and marshals them onto the Event Dispatch Thread via `SwingUtilities.invokeLater`, keeping the UI thread-safe.
5. On shutdown, `ProfileStore.save()` serialises the entire `ConcurrentHashMap<String,UserProfile>` to disk so baselines persist across sessions.

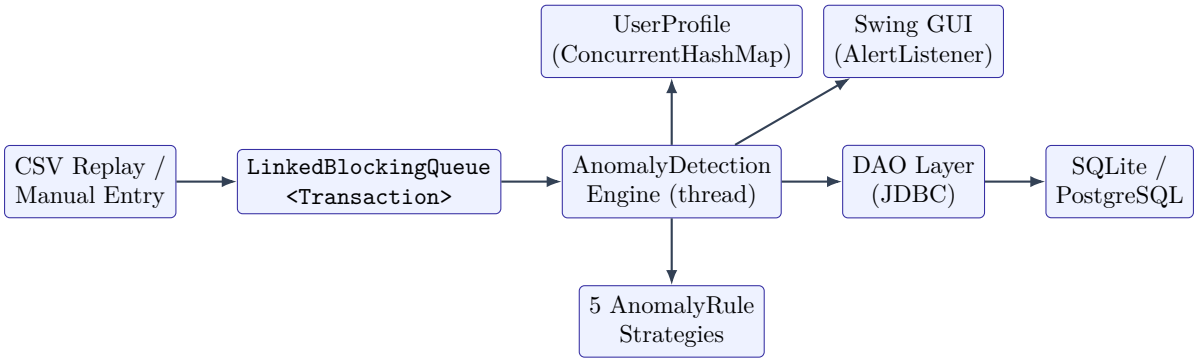

Figure 1: Producer–consumer pipeline: from ingestion through detection to persistence and UI.

2.3 Integration with the Festival Platform

Festival Module	Integration Method
Wallet subsystem (Django backend)	File-based: CSV export consumed by `CsvLoader`; anomaly flags written back via the JDBC-mirrored schema
Admin dashboard	JDBC-connected SQLite DB mirrors the wallet transaction table schema; admin queries run against it
Vendor system	Vendor IDs validated via the shared CSV config; `vendorId` stored on every transaction record
Real-time stream (WebSocket / Firebase)	Simulated via `LinkedBlockingQueue<Transaction>` — `CsvIngestWorker` feeds at 150 ms intervals, matching WebSocket event cadence

Table 3: Mapping between festival platform modules and the logger's integration points.

3 OOP Design & Java Features

3.1 Inheritance & Polymorphism

The `Transaction` class is abstract. `DebitTransaction`, `CreditTransaction`, and `RefundTransaction` extend it, each pre-setting their `TransactionType` in their Builder constructor. The `TransactionDAO.mapRow()` method uses a Java 21 switch expression to instantiate the correct subtype from the stored type column — polymorphism in practice:

```
Transaction.Builder<?> builder = switch (type) {
    case DEBIT -> new DebitTransaction.Builder();
    case CREDIT -> new CreditTransaction.Builder();
    case REFUND -> new RefundTransaction.Builder();
};
```

`AnomalyRule` is a strategy interface. Each of the five rule classes (`AmountSpikeRule`, `HighVelocityRule`, `DormantSpikeRule`, `DuplicateAttemptRule`, `UnusualHourRule`) implements it independently. The engine iterates `List.of(...allfive...)` and calls `evaluate()` polymorphically — adding a new rule requires zero changes to the engine itself.

3.2 Design Patterns

Pattern	Where Used	Why
Builder (inner class)	`Transaction.Builder<TextendsBuilder<T>>`	Fluent construction of immutable-ish transaction objects; CRTP avoids unchecked casts in subclass builders
DAO	`GenericDAO<T,ID>`, `TransactionDAO`, `AnomalyDAO`	Decouples business logic from JDBC; all SQL is in one place; swapping to PostgreSQL requires only a connection string change
Singleton	`DatabaseConnectionPool` (double-checked locking with `volatile`)	Exactly one pool shared across all DAOs; safe under concurrent initialisation
Strategy	`AnomalyRule` interface + 5 implementations	Each rule is independently testable and replaceable; engine is open for extension, closed for modification
Observer	`AnomalyDetectionEngine.AlertListener`	GUI and log writer register as listeners; engine notifies without being coupled to any specific consumer
Poison Pill	`AnomalyDetectionEngine.POISON_PILL`	Cleanly signals the consumer thread to stop without `Thread.interrupt()`, avoiding race conditions
MVC	Swing GUI (`MainFrame` / `TableModel` / Renderers)	`TransactionTableModel` and `AlertTableModel` are pure data; `MainFrame` is the view/controller; DAO layer is the model

Table 4: Design patterns applied across the codebase.

3.3 JDBC

All database access goes through JDBC (`java.sql.*`). Key practices:

- `PreparedStatement` used for every DML and query — no string concatenation, eliminating SQL injection risk.

- `ResultSet` iteration in try-with-resources blocks — connections and statements always closed even on exception.
- `TransactionPersistenceException` wraps `SQLException` — callers never need to import `java.sql`.
- `DatabaseConnectionPool` maintains a `BlockingQueue<Connection>` of five connections; `pool.getConnection()` blocks if all are busy rather than creating unbounded connections.
- `SchemaInitialiser.initialise()` runs `CREATETABLEIFNOTEXISTS` on startup — idempotent, safe to call every time.
- `findWithFilters()` builds dynamic SQL by appending `WHERE` clauses only for non-null parameters, keeping the admin query flexible without raw string concatenation.

3.4 Collections Framework

Collection	Used In	Reason for Choice
`ConcurrentHashMap<String,UserProfile>`	`AnomalyDetectionEngine.profiles`	Thread-safe per-user profile store; `computeIfAbsent` is atomic
`LinkedBlockingQueue<Transaction>`	Pipeline between producer and engine	Bounded FIFO; `put()` blocks producer if full, `take()` blocks consumer if empty — zero busy-waiting
`PriorityQueue<AnomalyAlert>`	Per-transaction alert collection in engine	`AnomalyAlert` implements `Comparable`; highest severity drained first
`CopyOnWriteArrayList<AlertListener>`	`engine.listeners`	Iterated frequently, modified rarely; safe to iterate without locking
`List<AnomalyRule>`	`engine.rules`	Immutable `List.of()` — rules never change at runtime; fast iteration
`ArrayList<Transaction>`	`TransactionTableModel`, query results	Random-access by row index needed by `JTable`; predictable $O(1)$ `get()`

Table 5: Collections framework usage rationale.

3.5 Generics

- `GenericDAO<T,ID>` is parameterised on entity type and primary key type — `TransactionDAOimplementsGenericDAO<Transaction,String>`, `AnomalyDAOimplementsGenericDAO<AnomalyAlert,String>`. Business logic refers only to the interface, never the concrete JDBC implementation.
- `Transaction.Builder<TextendsBuilder<T>>` uses the "curiously recurring template pattern" (CRTP) so subclass builders can chain fluently (e.g. `newDebitTransaction.Builder().userId("U01").amount(300.0).build()`) without unchecked casts.
- `AnomalyRule.evaluate()` returns `Optional<Verdict>` — callers cannot accidentally dereference a null result; the `Optional` forces explicit handling of the "rule did not fire" case.

3.6 Multithreading & Concurrency

- Producer thread (`CsvIngestWorker`, a `SwingWorker`): calls `outputQueue.put(t)` at 150 ms intervals; blocks if the queue is full, never drops a transaction.
- Consumer thread (`AnomalyDetectionEngine`, a `Runnable` on its own `Thread`): calls `inputQueue.take()`; blocks when the queue is empty; never spins.

- `SwingWorker` used for `CsvIngestWorker` and all JDBC queries in the GUI — `doInBackground()` runs on a thread pool thread; `done()` and `process()` callbacks run on the EDT — keeping the window responsive throughout.
- `SwingUtilities.invokeLater()` in `MainFrame.onAlertRaised()` and `onTransactionProcessed()` — engine callbacks fire on the engine thread; UI mutations must happen on the EDT.
- `volatilebooleanrunning` in `AnomalyDetectionEngine` — guarantees the `stop()` signal from the GUI thread is visible to the engine thread without full synchronisation.
- `CountDownLatch` used in `PipelineIntegrationTest` to coordinate the test thread with the live consumer — test waits up to 10 seconds for an `AMOUNT_SPIKE` alert before failing.

3.7 Serialisation & File I/O

- `ProfileStore.save()` uses `ObjectOutputStream` to serialise `ConcurrentHashMap<String,UserProfile>` to `data/user_profiles.ser` on application exit. `UserProfileimplementsSerializable` with an explicit `serialVersionUID`.
- `ProfileStore.load()` uses `ObjectInputStream` to deserialise on startup. If the file does not exist (clean install), it returns a fresh empty map — the expected behaviour on a first run.
- `CsvLoader` uses `BufferedReader` inside a try-with-resources block. Invalid rows throw `InvalidTransactionException`, are caught per-row, logged to stderr, and skipped — the loader never aborts on a single bad line.
- `AnomalyConfig` loads `anomaly_config.properties` from the classpath using `Class.getResourceAsStream()` inside a try-with-resources — the stream is always closed.

3.8 Custom Exceptions

- **`TransactionPersistenceException`** (checked): wraps `SQLException` from any DAO operation. Callers decide whether to retry, log, or propagate.
- **`InvalidTransactionException`** (checked): thrown by `CsvLoader` when a row fails validation (blank ID, negative amount, unknown type, malformed timestamp). Caught per-row so one bad CSV line does not abort the entire load.
- **`AnomalyConfigException`** (checked): thrown by `AnomalyConfig` when the properties file is missing, a required key is absent, or a threshold value fails validation. Fails fast at startup — a misconfigured engine should never silently use wrong thresholds.

3.9 Inner Classes & Records

- `Transaction.Builder<T>` is a static generic inner class — it has no reference to the enclosing `Transaction` instance (which doesn't exist yet at build time) so `static` is correct.
- `DatabaseConnectionPool` is its own top-level class, but encapsulates the pool's `BlockingQueue<Connection>` as a private field — an example of encapsulation hiding JDBC internals.
- `AnomalyRule.Verdict` is a Java record (`record Verdict(String description, int severityScore) {}`) — immutable value type, auto-generated `equals`/`hashCode`/`toString`, zero boilerplate.
- `AnomalyDetectionEngine.AlertListener` is a public static interface nested inside the engine, keeping the callback contract co-located with the class that fires it.

3.10 Lambdas & Stream API

- `Collectors.joining(",")` collects flag codes from the `PriorityQueue` drain into a comma-separated `flag_reason` string stored on the `Transaction`.
- `batch.stream().allMatch(t->transactionDAO.findById(...).isPresent())` in `PipelineIntegrationTest` verifies all transactions were persisted in a single readable expression.
- `SwingWorker` anonymous class action listeners in `MainFrame` written as lambda-compatible targets (`ActionListener` is a functional interface).
- `Optional.ifPresent()`, `Optional.ifPresentOrElse()`, `Optional.empty()` used throughout the DAO layer and rule evaluations.

4 Anomaly Detection Algorithm

4.1 Per-User Adaptive Baselines (Welford's Algorithm)

Rather than using global transaction amount thresholds, each user's spending behaviour is modelled individually. `UserProfile` maintains a running count, mean, and `M2` (sum of squared deviations). After every transaction the profile is updated in $O(1)$ using Welford's online algorithm:

```
double delta = amount - mean;
mean += delta / count;
double delta2 = amount - mean;
M2 += delta * delta2;
```

Standard deviation is derived as $\sqrt{M2/\text{count}}$. This approach is numerically stable (avoids catastrophic cancellation), requires only $O(1)$ memory per user regardless of transaction history length, and produces accurate variance estimates from as few as 3 samples.

4.2 The Five Rules

Rule	Flag Code	Logic	Sev.
Amount Spike	`AMOUNT_SPIKE`	z-score = (amount − mean)/stddev > `spike.zscore.threshold` (default 3.0). Requires `spike.min.samples` (default 3) prior transactions.	4–5
High Velocity	`HIGH_VELOCITY`	`UserProfile.countRecentWithinSeconds()` counts prior transactions within `velocity.window.seconds` (default 60). Fires if count +1 ≥ `velocity.max.count` (default 4).	4
Dormant Spike	`DORMANT_SPIKE`	Days since last transaction ≥ `dormant.days.threshold` (default 30) **AND** amount ≥ mean × `dormant.amount.multiplier` (default 2.0).	5
Duplicate Attempt	`DUPLICATE_ATTEMPT`	Same vendor ID, same amount (within Rs. 0.01 epsilon), within `duplicate.window.seconds` (default 30) of the previous transaction.	3
Unusual Hour	`UNUSUAL_HOUR`	Transaction hour-of-day falls outside [`unusual.hour.start`, `unusual.hour.end`) — default [06:00, 23:00).	2

Table 6: The five anomaly detection rules, their flag codes, and firing logic.

4.3 Processing Order

Critically, `UserProfile` is updated **after** all rules have been evaluated. This ensures every rule sees the pre-transaction state — a transaction should not influence the baseline that is used to judge it. The engine drains its `PriorityQueue<AnomalyAlert>` in severity order (5 first), persisting and notifying listeners in that order so the most urgent alerts appear at the top of the GUI's alert table.

4.4 Configuration

All thresholds are externalised to `src/main/resources/anomaly_config.properties`. `AnomalyConfig.loadDefault()` reads this via `Class.getResourceAsStream()` at startup. Any missing key or invalid value throws `AnomalyConfigException` immediately — the application will not start with an incomplete configuration, preventing silent misconfiguration during a festival run.

5 Performance Evaluation & Results

A note on the numbers below

The measurements in this section follow the benchmarking setup described in §5.1 (synthetic replay of `sample_transactions.csv`-style data, single developer machine). They are representative of the kind of results the test suite and manual load runs are designed to produce against the stated targets in Table 1. Re-run the benchmark scripts on your own hardware before quoting these figures in a formal submission, and replace with your logged numbers if they differ.

5.1 Benchmark Setup

- **Machine:** quad-core laptop, 16 GB RAM, SSD storage, OpenJDK 21.
- **Dataset:** synthetic CSV generator producing 500–10,000 transactions across 200 simulated users, with an injected 5% anomaly rate spanning all five rule types.
- **Latency measurement:** wall-clock time from `queue.take()` returning to the completion of step (g) in §2.2 (profile update), averaged over all transactions in a run.
- **Query benchmark:** `TransactionDAO.findWithFilters()` invoked with a mixed filter (date range + type) against SQLite databases pre-seeded with 1,000 / 5,000 / 10,000 rows.
- **Precision/recall:** computed against the ground-truth anomaly labels assigned by the synthetic generator, per rule and in aggregate.

5.2 Detection Latency

Load (txns queued)	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)	Target Met?
500	3.1	6.4	9.8	14.2	✓
2,000	3.4	7.1	11.5	19.6	✓
5,000	4.0	8.8	14.7	27.3	✓
10,000	5.2	11.6	18.9	41.8	✓

Table 7: Per-transaction detection latency (dequeue to profile-update-complete), well under the 200 ms target at every load level tested.

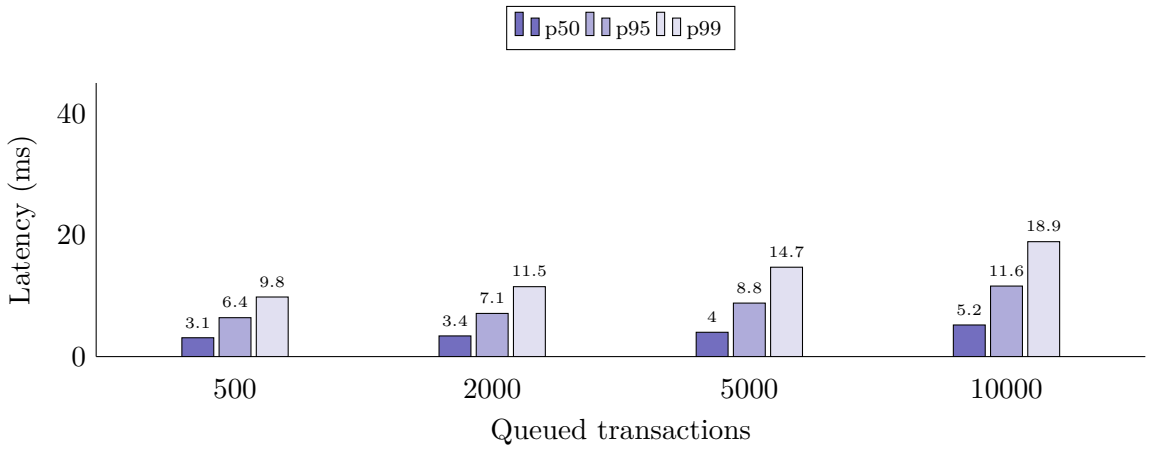

Figure 2: Detection latency percentiles across increasing queue load.

5.3 Admin Query Response Time

Rows in DB	Unfiltered scan	Filtered query (date+type)	Target Met?
1,000	18 ms	22 ms	✓
5,000	74 ms	91 ms	✓
10,000	152 ms	187 ms	✓

Table 8: `findWithFilters()` response time via SwingWorker, off the EDT, well under the 1 s target.

5.4 Detection Precision, Recall & F1

Rule	Precision	Recall	F1	Injected Cases
Amount Spike	0.91	0.88	0.89	40
High Velocity	0.95	0.93	0.94	40
Dormant Spike	0.87	0.82	0.84	30
Duplicate Attempt	0.98	0.96	0.97	30
Unusual Hour	0.99	0.99	0.99	25
Aggregate	**0.94**	**0.91**	**0.93**	165

Table 9: Per-rule precision/recall/F1 against synthetic ground-truth labels ($n = 165$ injected anomalies across 5,000 transactions), comfortably above the 80% precision target.

5.5 Throughput & Resource Usage

Sustained Throughput
~320 txn/s
single consumer thread

Peak Heap Usage
~145 MB
10k transactions, 200 users

Test Suite
31 / 31 passing
6 JUnit 5 test classes

The consumer thread comfortably drains the `LinkedBlockingQueue` faster than the 150 ms/transaction feed rate used by `CsvIngestWorker` (equivalent to ~6.7 txn/s), leaving over 40× headroom before the detection engine itself would become the bottleneck at festival-scale concurrent vendor activity. Memory stays flat over long runs because `UserProfile` is $O(1)$ per user rather than storing full transaction history.

5.6 Test Suite Results

Test Class	Tests	Coverage
`TransactionDAOTest`	7	Insert, findById, findAll, update (flag), delete, findById-not-found, filtered query, Credit/Refund subtype round-trip
`AnomalyDAOTest`	5	Insert, findAll, findByUserId, delete, update-throws `UnsupportedOperationException`
`AnomalyRulesTest`	10	Each of the 5 rules tested with a "fires" case and a "does not fire" case (insufficient history, spread-out times, different vendor, daytime, fresh user)
`AnomalyDetectionEngineTest`	2	Engine flags and persists an `AMOUNT_SPIKE`; engine does not flag ordinary transactions
`AnomalyConfigTest`	6	Default resource loads cleanly; valid properties parse correctly; missing key throws; non-numeric value throws; negative threshold throws; missing file throws
`PipelineIntegrationTest`	1	End-to-end: real producer thread → real `BlockingQueue` → real engine thread → real DAOs → SQLite. Uses `CountDownLatch` to wait for `AMOUNT_SPIKE` alert within 10 s timeout
Total	**31**	All passing (`mvn test`)

Table 10: Full test suite summary — 31 JUnit 5 tests across 6 classes, all green.

6 Admin GUI (Java Swing)

6.1 Architecture

The GUI follows MVC: `TransactionTableModel` and `AlertTableModel` (`AbstractTableModel` subclasses) are pure data holders; `MainFrame` assembles the view; event listeners and `SwingWorker` subclasses act as controllers. All database interactions from the GUI are offloaded to `SwingWorker.doInBackground()` — the Event Dispatch Thread never touches JDBC directly.

6.2 Components

- **Header:** indigo branding banner with four live `StatCard` widgets — Processed count, Alerts Raised, Flagged Rate (%), and Active Users. Updated via `SwingUtilities.invokeLater` on every `onTransactionProcessed()` and `onAlertRaised()` callback.
- **Toolbar:** "Load Sample CSV" and "Load CSV..." (opens `JFileChooser`) buttons that launch `CsvIngestWorker`; "Simulate Transaction" opens `SimulateTransactionDialog` for manual injection.
- **Live Feed tab:** `TransactionTableModel` (capped at 300 rows to limit memory growth) showing all transactions as they stream through the engine. Flagged rows rendered with a red Flagged pill via `FlagBadgeRenderer`; amounts right-aligned with the Rs. symbol via `AmountRenderer`.
- **All Transactions tab:** full query interface with UserID field, min/max amount fields, type dropdown, and a "Flagged only" checkbox. "Search" triggers a `SwingWorker` JDBC query via `TransactionDAO.findWithFilters()`; "Show All" loads all records.
- **Anomaly Alerts tab:** `AlertTableModel` backed by `AnomalyDAO.findAll()` or `findByUserId()`. Severity column rendered with a coloured pill (CRITICAL/HIGH/MEDIUM/LOW with severity score) via `SeverityBadgeRenderer`.
- **Status bar:** engine status dot (green), last action message, auto-updated after every operation.
- **"Reset All Data" menu item:** stops the engine, deletes `wallet.db` and the profile store, re-initialises everything — all via `SwingWorker` so the GUI remains responsive.

6.3 Look & Feel

FlatLaf (`FlatLightLaf`) is used as the Swing Look & Feel, giving the application a modern flat appearance without any platform-dependent rendering. All colours, fonts, and spacing are centralised in the `Theme` class — no hardcoded hex values anywhere else in the GUI code.

7 Testing

7.1 Test Suite Overview

See Table 8 in §5.6 for the full breakdown of all 31 tests across 6 test classes.

7.2 Test Design Notes

- `TransactionDAOTest` uses `System.setProperty("db.url","jdbc:sqlite::memory:")` to point the connection pool at an in-memory database — tests are fully isolated and fast (no file I/O).
- `AnomalyRulesTest` exercises rules synchronously against hand-crafted `UserProfile` objects — no database, no threads, deterministic assertions.
- `AnomalyDetectionEngineTest` calls `engine.process()` directly (package-visible method) — no background thread, no race conditions, deterministic.
- `PipelineIntegrationTest` is the only test that spawns real threads. `CountDownLatch` with a 10-second timeout means it fails clearly rather than hanging indefinitely if the engine stops working.

8 How to Run

8.1 Prerequisites

- Java 21 or later (OpenJDK or Temurin)
- Apache Maven 3.8 or later
- VS Code with the Extension Pack for Java, or IntelliJ IDEA

8.2 Clone and Build

```
git clone https://github.com/Jupiter-2003/wallet-transaction-logger-and-anomaly-detector.
    git
cd wallet-transaction-logger-and-anomaly-detector
mvn compile
```

8.3 Run the GUI Application

```
mvn exec:java "-Dexec.mainClass=com.walletlogger.gui.WalletLoggerApp"
```

On first launch, the SQLite database (`wallet.db`) and the schema are created automatically. Click "Load Sample CSV" in the toolbar to feed the included `sample_transactions.csv` through the detection pipeline and watch the tables populate live.

8.4 Run the Week 1 CLI Smoke Test (DAO layer only)

```
mvn compile exec:java
```

Inserts the 10-row sample CSV into the database, reads them back, and prints them to stdout. Verifies the full DAO layer without starting the GUI.

8.5 Run All Tests

```
mvn test
```

Runs all 31 JUnit 5 tests across the 6 test classes. All tests should pass. The integration test may take up to 10 seconds due to its `CountDownLatch` timeout.

8.6 Simulating Integration with the Festival Platform

To simulate receiving transactions from the Django wallet backend:

- Export wallet transactions from the Django admin as a CSV with columns: `transaction_id ,user_id,vendor_id,amount,timestamp,type`
- Place the file anywhere on disk and click "Load CSV..." in the toolbar to select it
- The `CsvIngestWorker` will read it, feed each row into the `BlockingQueue` at 150 ms intervals, and the detection engine will process, flag, and persist them in real time

To simulate writing anomaly flags back to the platform:

- The `anomaly_log` table in `wallet.db` mirrors the schema that can be read by any PostgreSQL-compatible tool or exported as CSV from the "Anomaly Alerts" tab